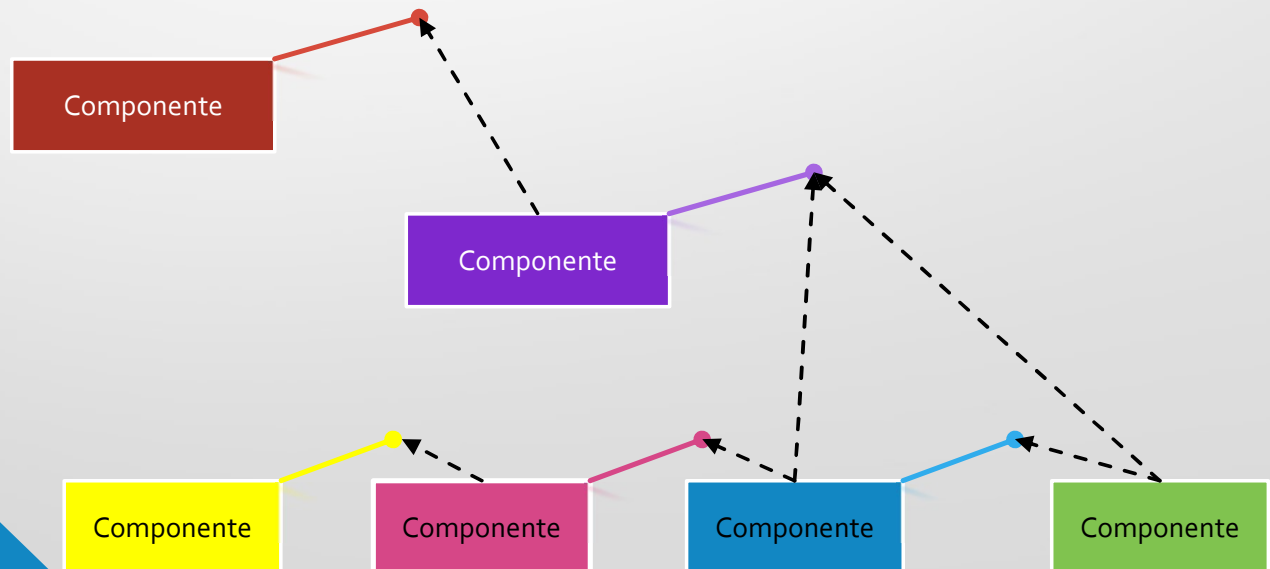


Tópico 2:

Engenharia de Software Baseada em Componentes ESBC



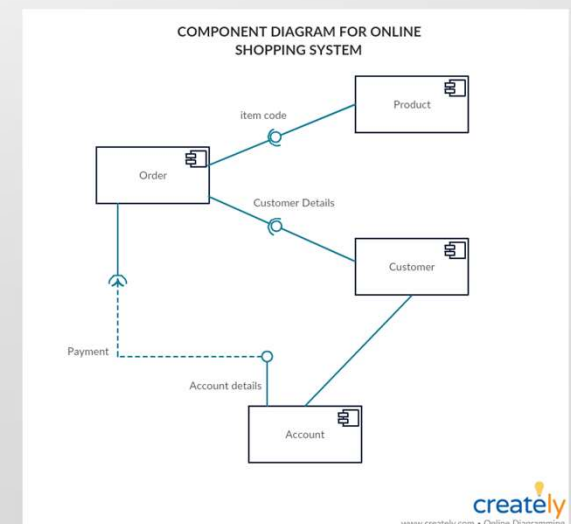
Objetivos da Aula:

1. Definir o que é a ESBC
2. Entender o que é um componente.
3. Descrever as características dos componentes.
4. Definir o que é um modelo de componente.
5. Estudar uma abordagem para identificação de componentes.



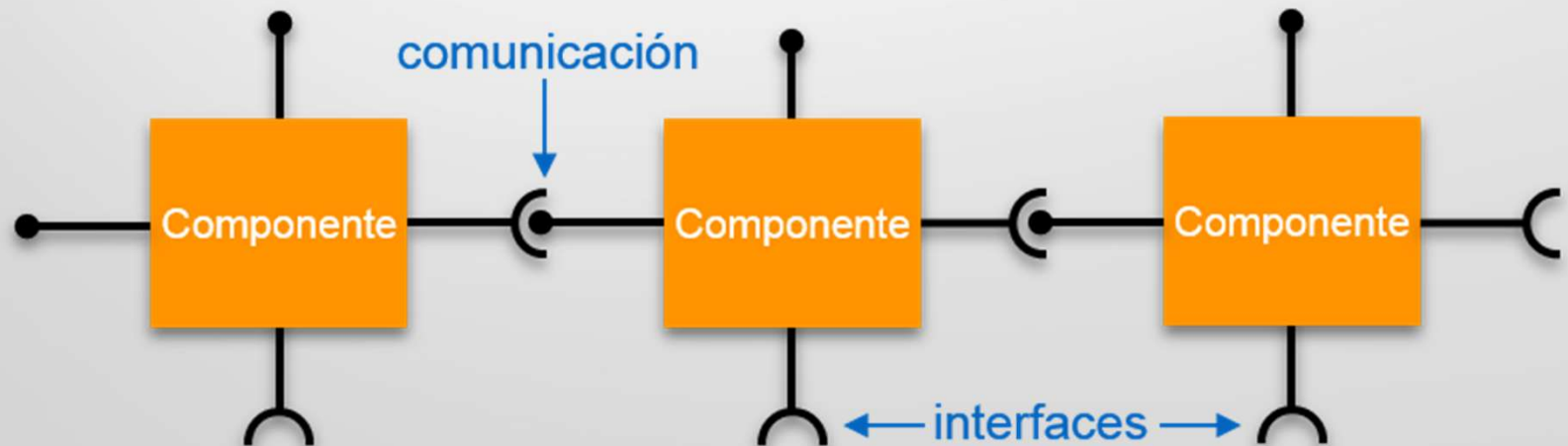
O que é ESBC?

- ❑ ***A Engenharia de Software baseada em Componentes ESBC***, é o processo de definir, implementar, integrar ou compor componentes independentes, pouco acoplados em sistemas.

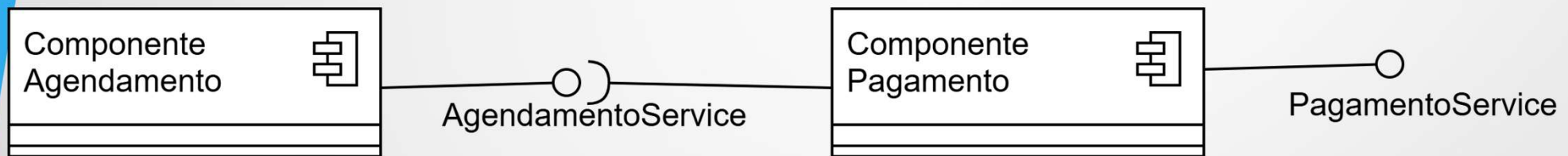


O que são Componentes?

- Um componente de software é uma **unidade de software independente, reutilizável e substituível**, que **encapsula sua implementação e expõe suas funcionalidades** por meio de interfaces bem definidas.



O que são Componentes?



Características do Componentes

1. **Encapsulamento:** a lógica interna é escondida.
2. **Interfaces bem definidas:** descrevem o que é oferecido e o que é requerido.
3. **Reuso:** projetado para ser usado em múltiplas aplicações.
4. **Independente:** desenvolvido e implantado isoladamente.
5. **Composição:** sistemas são montados pela integração de componentes.

Interfaces de Componentes

- Uma **interface** é o **contrato** que descreve como o componente pode ser usado ou como ele interage com outros componentes.

Interface “requer”

Define os serviços necessários que devem ser fornecidos por outros componentes.



Componente



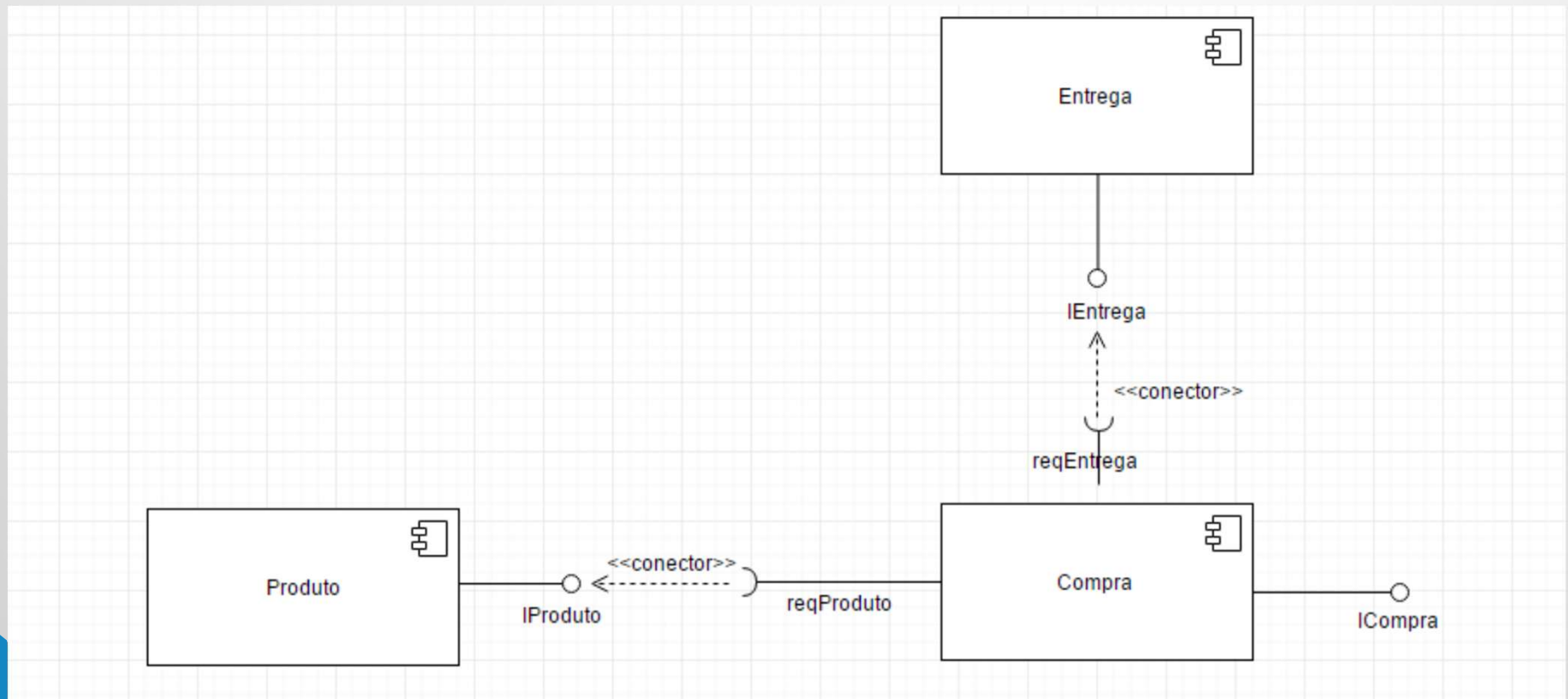
Interface “fornece”

Define os serviços fornecidos pelo componente para outros componentes.

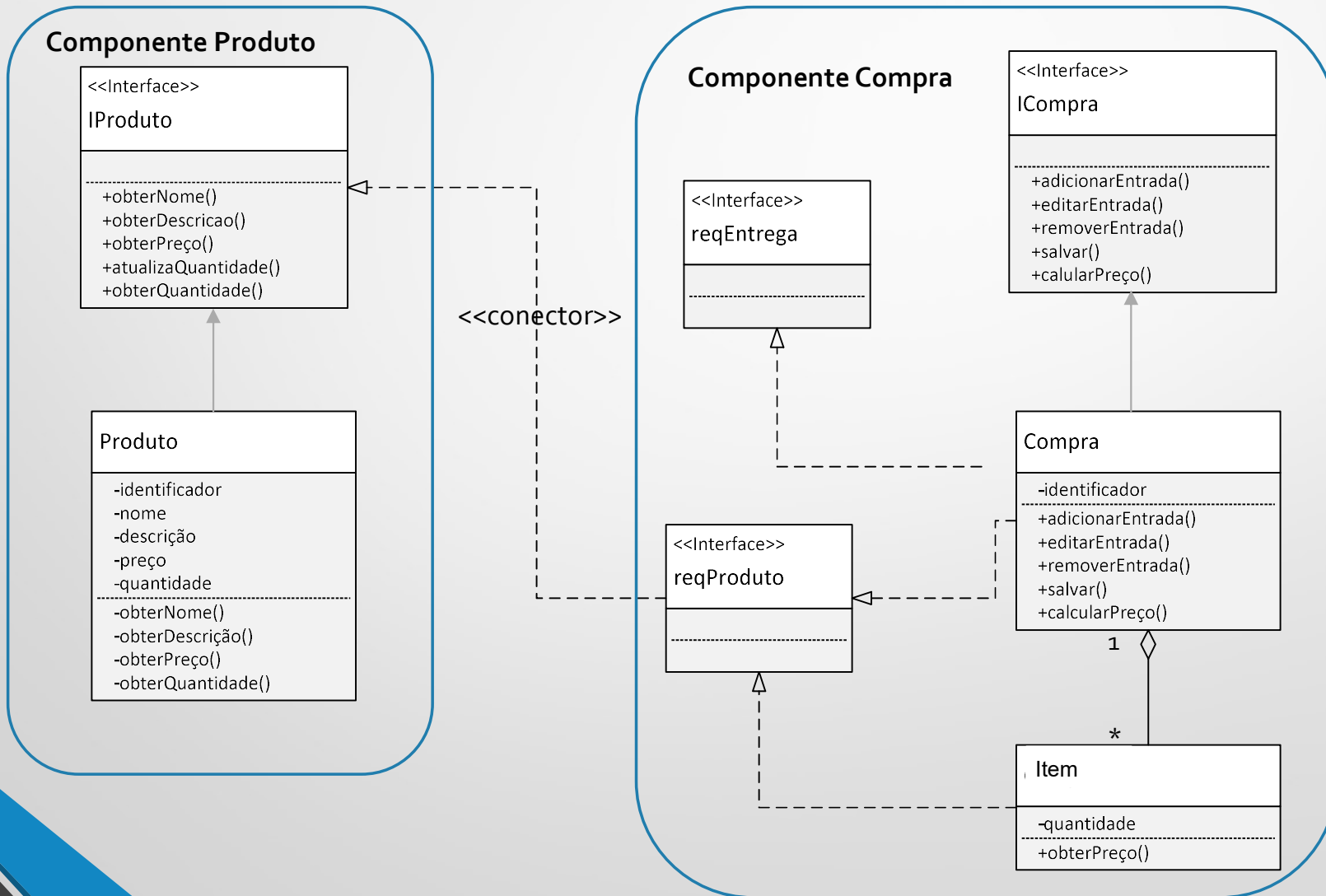


Exemplo de Componentes

Componentes da Livraria Virtual



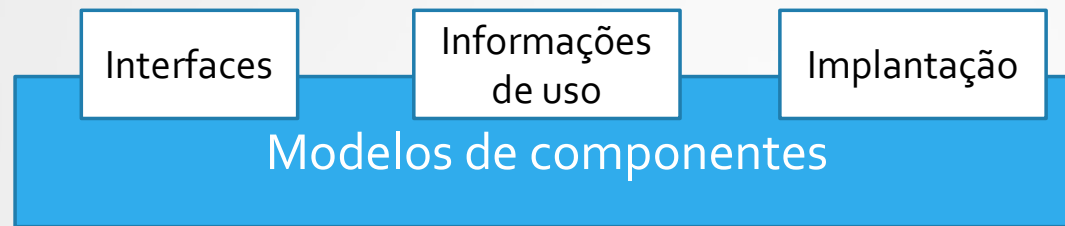
Interface de Componentes



Modelos de Componentes

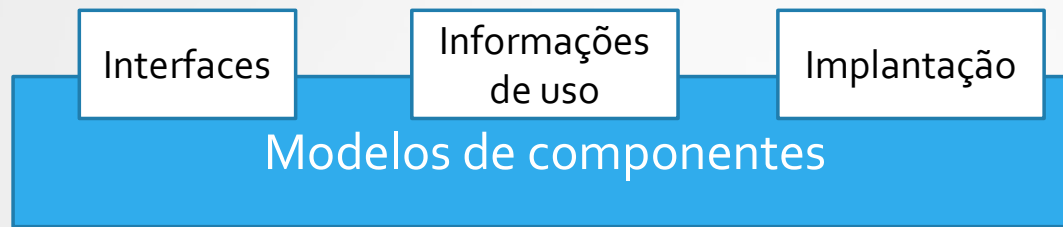
- É o conjunto de regras que define como componentes são construídos, como se conectam, como expõem interfaces e como são implantados.
- Garantem a interoperabilidade entre os componentes.
- Modelos mais usados:
 - **CORBA** ((Common Object Request Broker Architecture): foco em middleware e interoperabilidade.
 - **JavaBeans / Enterprise JavaBeans**: reuso em aplicações corporativas, contêineres, ciclo de vida.
 - **.NET (Component Object Model, assemblies)**: focado na integração dentro do ecossistema Windows.

Elementos de um modelo de Componentes



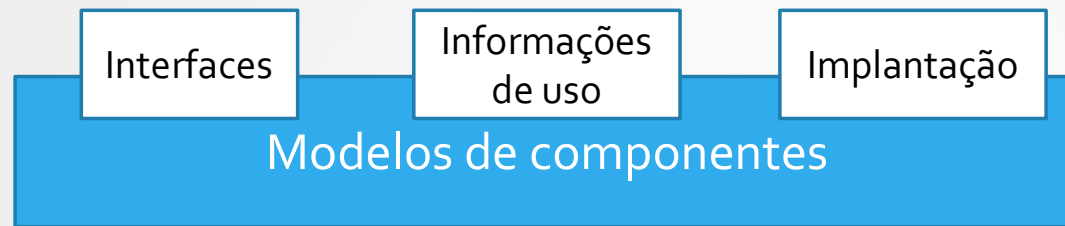
- Interface: contrato:
 - ✓ Nome da operação
 - ✓ Parâmetros e exceções;
 - ✓ Linguagem usada para especificar as interface;

Elementos de um modelo de Componentes



- Informações de uso: para que o Componente funcione corretamente:
 - ✓ Localização URI,
 - ✓ Metadados: interfaces e atributos; quais serviços são providos e requeridos;
 - ✓ Informações de configuração do componente para um sistema de aplicação.

Elementos de um modelo de Componentes



- Implantação:

- Empacotamento dos componentes para implantação como rotinas executáveis e independentes.
- Regras de governança para alterar e substituir componentes.
- Documentação do componentes.

Processos de ESBC

- Desenvolvimento para reuso:
 - Desenvolvimento de componentes que serão reusados em outras aplicações.
- Desenvolvimento com reuso:
 - Desenvolvimento de novas aplicações usando componentes e serviços existentes.

Processos de ESBC

Aspecto	Desenvolvimento com reuso	Desenvolvimento para reuso
Foco	Aproveitar componentes prontos	Criar componentes reutilizáveis
Objetivo	Reduzir tempo e custo de desenvolvimento	Aumentar produtividade futura
Atividades-chave	Buscar, selecionar, adaptar, integrar	Generalizar, projetar, documentar, empacotar
Exemplo	Usar API de pagamento do PayPal	Criar módulo de pagamento genérico para vários apps
Benefício	Rapidez imediata	Sustentabilidade e economia em longo prazo

Desenvolvimento COM REÚSO

1. Identificação de
necessidades



2. Busca e seleção
de componentes



3. Adaptação
e integração



4. Composição
do sistema



5. Teste e validação

Desenvolvimento PARA REÚSO

1. Análise de domínio



2. Generalização e
abstração



3. Definição de
interfaces claras



4. Implementação modular
e independente



5. Empacotamento e
documentação

Desenvolvimento para Reuso



Processos de ESBC

- **Abordagem de Cheesman e Daniels**

Ajudar a responder três perguntas fundamentais:

- **Quais são os componentes do sistema?**
- **Quais interfaces cada componente oferece ou requer?**
- **Quais operações fazem parte dessas interfaces e sob quais contratos?**

Processos de ESBC

- **Abordagem de Cheesman e Daniels**

Processo composto pelas seguintes atividades:

1. Requisitos – Casos de Uso
2. Interface do Sistema
3. Identificação de Interfaces
4. Identificação de Componentes
5. Contratos das Operações
6. Dependências entre Componentes
7. Diagrama final consolidado

Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

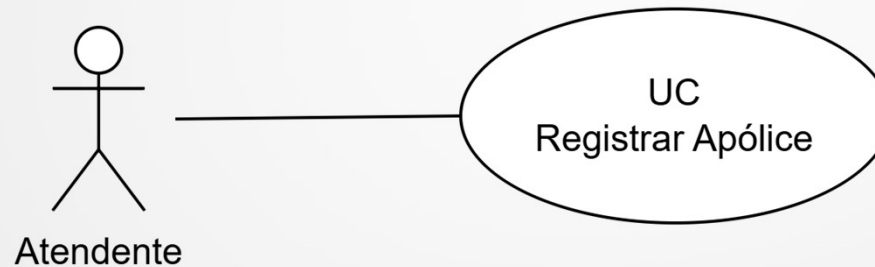
Objetivo: Permitir que um atendente registre uma nova apólice de seguro para um cliente.



Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

1. Requisito – Casos de Uso



Fluxo Principal

1. Atendente informa CPF do cliente
2. Sistema valida cliente
3. Atendente informa dados do veículo
4. Sistema calcula prêmio
5. Sistema gera apólice
6. Sistema registra pagamento inicial

Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

2. Identificação da Interface do Sistema

<<interface>>

SistemaSeguro

- + validarCliente(cpf)
- + calcularPremio(dadosVeiculo)
- + registrarApolice(dadosCliente, dadosVeiculo)
- + registrarPagamento(numeroApolice, valor)

Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

3. Agrupamento de Operações em Interfaces Coesas

Quais operações manipulam o mesmo conceito de negócio?

```
<<interface>>  
ClienteService
```

```
-----  
+ validarCliente(cpf)
```

Cenário do Sistema

<<interface>>

PremioService

+calcularPremio(dadosVeiculo)

<<interface>>

ApoliceService

+ registrarApolice(dadosCliente, dadosVeiculo)

<<interface>>

PagamentoService

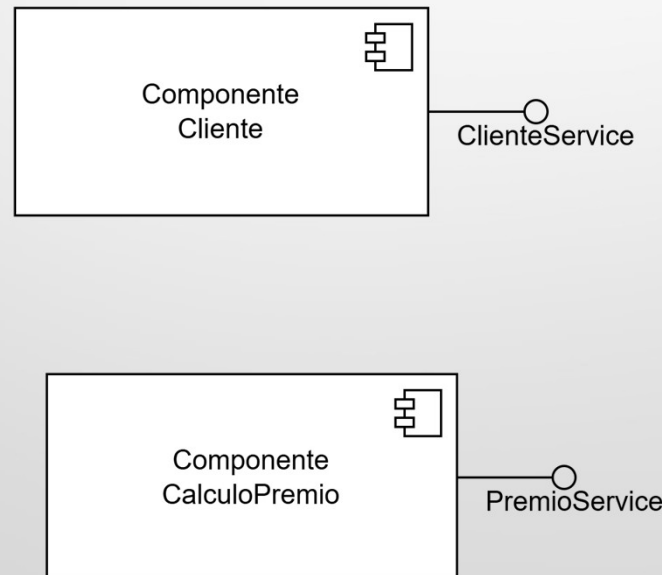
+ registrarPagamento(numeroApolice, valor)

Cenário do Sistema

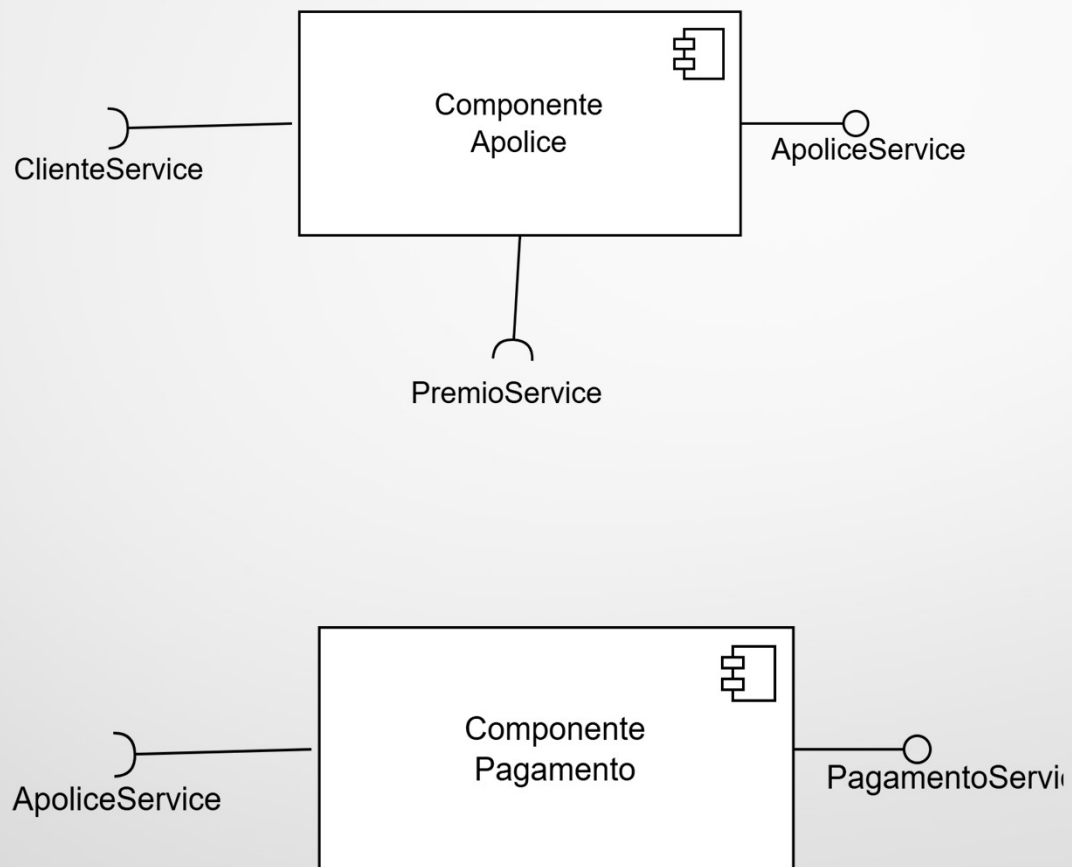
Sistema: **Gestão de Apólices de Seguro Auto**

4. Identificação de Componentes

Agora cada interface deve ser realizada por um componente.



Cenário do Sistema



Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

5. Definição de Contratos

Agora formalizamos as operações.

Interface ClienteService

Operação: validarCliente(cpf)

Pré-condição:

- CPF informado não pode ser nulo
- CPF deve ter formato válido

Pós-condição:

- Cliente válido confirmado
ou
- Cliente não encontrado

Cenário do Sistema

Interface PremioService

Operação: calcularPremio(dadosVeiculo)

Pré-condição:

- Veículo deve possuir ano válido
- Categoria do veículo definida

Pós-condição:

- Valor do prêmio calculado
- Valor retornado maior que zero

Cenário do Sistema

Interface ApoliceService

Operação: registrarApolice(dadosCliente, dadosVeiculo)

Pré-condição:

- Cliente validado
- Prêmio já calculado

Pós-condição:

- Apólice criada com status "Ativa"
- Número único gerado
- Apólice persistida

Cenário do Sistema

Interface PagamentoService

Operação: registrarPagamento(numeroApolice, valor)

Pré-condição:

- Apólice deve existir
- Valor deve ser igual ou maior que valor mínimo

Pós-condição:

- Pagamento registrado
- Status da apólice atualizado se necessário

Cenário do Sistema

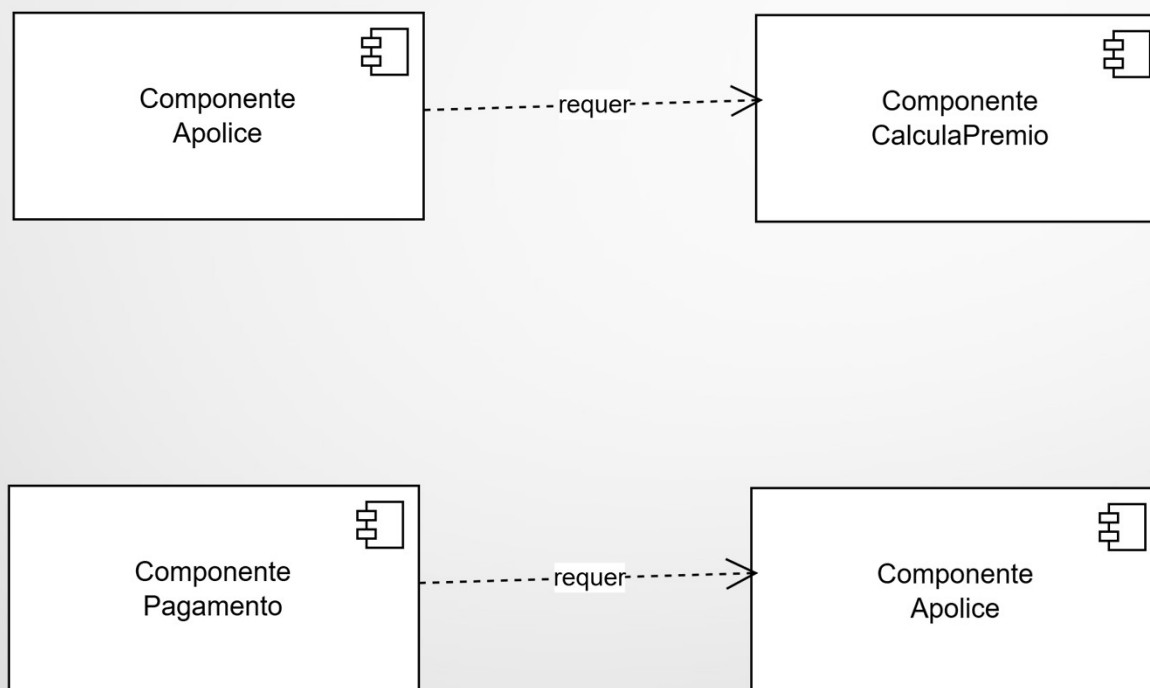
Sistema: **Gestão de Apólices de Seguro Auto**

6. Modelagem das Dependências

Agora conectamos os componentes



Cenário do Sistema



Cenário do Sistema

Sistema: **Gestão de Apólices de Seguro Auto**

7. Diagrama Componentes

